

What is a function ?

In C++, a function is a **group of statements** which upon **invocation** or **calling** performs a **task**. In general, you can compare them with mathematical functions

1. name
eg:1 $f(x) = x^2$
 2. Zero or more args
eg:2 $g(x, y) = x^2 + y^2$
 3. task
 4. You can ret 0 or 1 val.
- $f(2) \rightarrow 4$
 $f(4) \rightarrow 16$
 $g(3, 4) \rightarrow 25$

```
#include<iostream>

using namespace std;

int main() {
    // ...

    return 0; // ret. stmt
}
```

fn design

```
/*
return_type fn_name(args) {
    // ...

    return_stmt
}
*/
```

fn signature

return; $\Rightarrow$ write this when ret. type of the fn is void

optional

➤ A function to print a **greeting** message

```
void greet ( ) {
    cout << "hello world";
    return; // optional
}
```

```

    cout << "hello world";
    return; // optional
}

```

How to call / invoke a function ?

```

main() {
    ...
    → greet( ); // fn invocation / fn call
    ...
}

```

➤ A function to check if a given integer is even

formal parameter

```

void isEven(int n) {
    if (n % 2 == 0) {
        cout << "even";
    } else {
        cout << "odd";
    }
}

```

copy

```

main() {
    ⇒ isEven(5); // arg. / actual parameter
    ⇒ isEven(2); // arg.
}

```

copy

Diagram showing the flow of data: A box labeled 'n' in the function definition is connected by an arrow labeled 'copy' to a box labeled 'n' in the function call. Another arrow labeled 'copy' points from the argument '5' in the function call to the parameter 'n' in the function definition.

```

bool isEven(int n) {
    if (n % 2 == 0) {
        return true;
    } else {
        return false;
    }
}

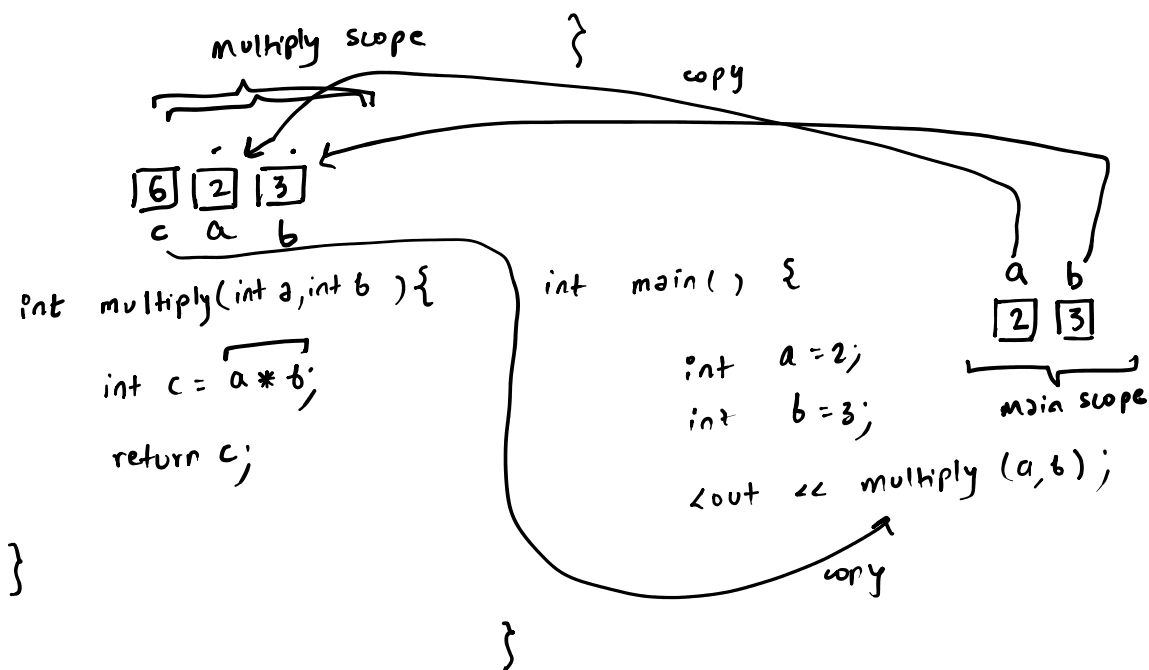
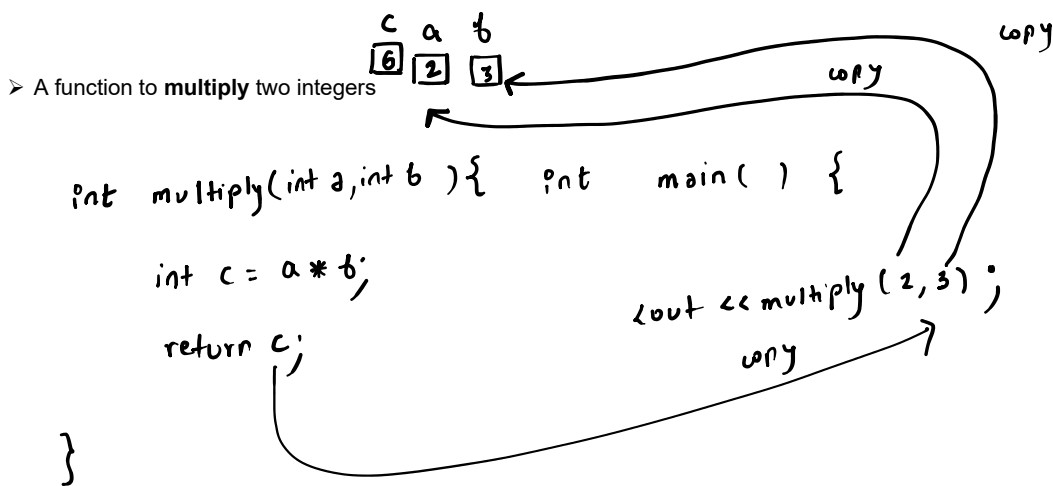
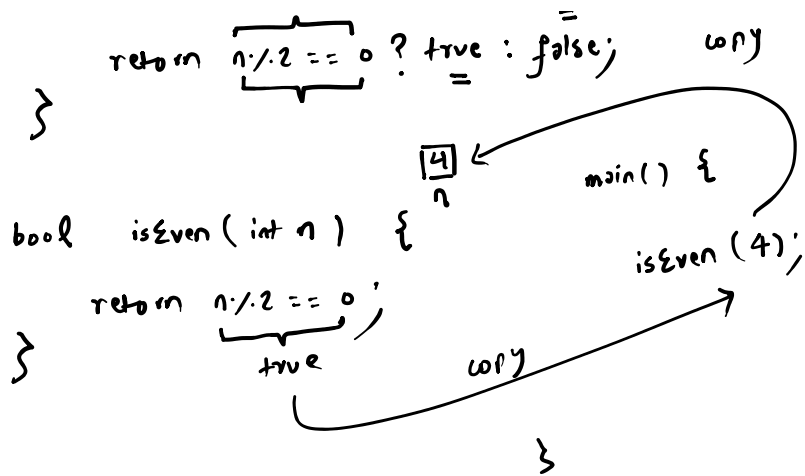
bool isEven(int n) {
    if (n % 2 == 0) {
        return true;
    }
    return false;
}

```

```

bool isEven(int n) {
    return n % 2 == 0 ? true : false; // copy
}

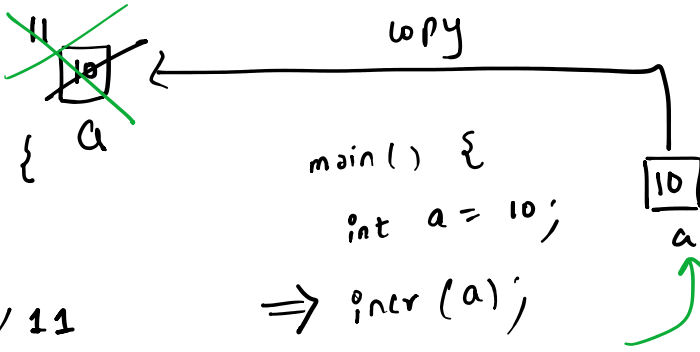
```



> A function to **increment** an integers

```
void incr (int a) {
    a++;
    <out << a; // 11
}
```

```
main() {
    int a = 10;
    => incr(a);
    <out << a;
}
```



// pass by value

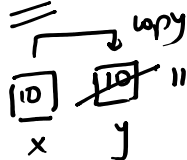
we are sending copy
of the arg. to the fn
∴ any changes done to
parameter will not be
reflected in the arg.
w/ the caller fn i.e main()

Reference Variables (lvalue Reference)

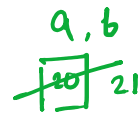
A reference variable (lvalue reference) is an **alias** for an existing variable (lvalue).

⇒

```
int x = 10;
int y = x;
y++;
<out << x; 10
```



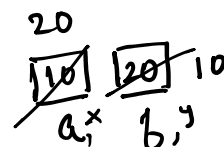
```
int a = 20;
int b = a;
b++;
<out << a; // 21
```



> A function to **swap** two integers

```
void myswap(int& x, int& y) {
    int temp = x;
    x = y;
    y = temp;
}
```

```
main() {
    int a = 10;
    int b = 20;
}
```



```
void myswap(int& x, int& y) {
    int temp = x;
    x = y;
    y = temp;
}
```

u, 0,

```
int a=10;
int b=20;
cout << a << b; // 10 20
=> myswap(a, b);
cout << a << b; // 20 10
```

- A reference variables must be **initialized**.
- A reference variables cannot be **reseated**.
- A reference variables cannot be assigned the **NULL** value.
- A reference can be also be **const**

```
int x = 10;
const int &y = x;
```

```
cout << y; // 10
```

```
y++; // error since y is a const ref
```

```
x++; // works since x is a normal var
```

```
const int a = 50;
const int &b = a;
cout << a << b; // 50 50
b++; // error since b is a const ref
a++; // error since a is a const var
```

```
const int c = 100;
int &d = c;
```

can you create a non-const ref to a const var?
you cannot do this

```
int x = 10;
int &y = x;
```

```
y++;
```

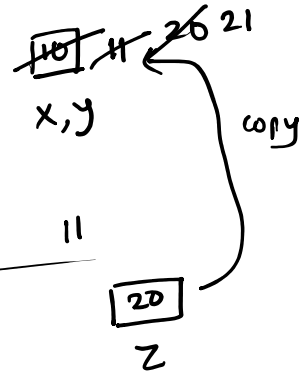
```
cout << x << y; // 11 11
```

```
int z = 20;
```

```
y = z;
```

```
y++;
```

```
cout << x << y << z;
// 21 21 20
```



- A function to check if an integer is **prime**

$$\underbrace{2 \text{ to } n-1}$$

cmath
↓
sqrt(.....)

$$\underbrace{2 \text{ to } \sqrt{n}}$$

- A function to print all the **prime numbers** in the range 2 to **M**

2, 3, 5, 7, 11, 13, 17, 19
m = 20

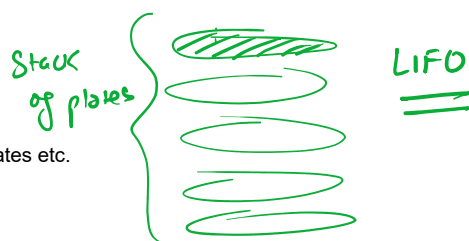
m = 20

2	3	4	5	6	7		17	18	19	20
✓	✓	x	✓	x	✓			x	✓	x

Imp

Function Call Stack

A **stack** is a pile of objects - a stack of CDs, books, plates etc.



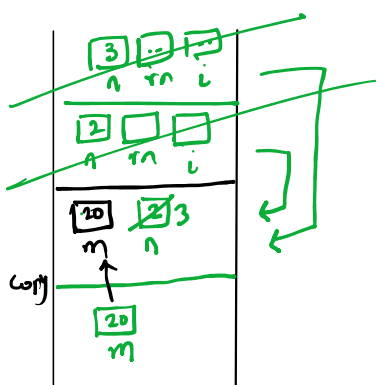
⇒

isPrime

isPrime

printPrime

main



f_n(call) 8+k

★★

Passing Arrays to Functions

```

int computeSum(int arr[], int n) {
    int sum = 0;
    for(int i=0; i<n; i++) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr)/sizeof(int);
    cout << computeSum(arr, n) << endl;
    return 0;
}

```

In C++, arrays are by default passed **by reference**.

➤ Return by Value vs Return by Reference

ret. by value

```

int multiply(int a, int b) {
    int c = a * b;
    return c;
}

int main() {
    cout << multiply(2, 3) << endl;
    return 0;
}

```

Handwritten notes:
 - Variables `a`, `b`, and `c` are boxed with values 2, 3, and 6 respectively.
 - Arrows labeled "copy" point from the boxed `a` and `b` to the boxed `c`.
 - A bracket under `multiply(2, 3)` in `main` is labeled "copy".
 - The output `// 6` is shown.

```

int multiply(int a, int b) {
    int c = a * b; // local variable
    return c;
}

int main() {
    cout << multiply(2, 3) << endl;
    return 0;
}

```

Handwritten notes:
 - Variables `a`, `b`, and `c` are boxed with values 2, 3, and 6 respectively.
 - A bracket under `multiply(2, 3)` in `main` is labeled "copy".
 - The output `// 6` is shown.
 - A green arrow points from the `copy` label to the word "undefined" written below.
 - A green note says "Dangling Ref. Problem".

ret. by ref.

```

int c; // global variable

int multiply(int a, int b) {
    c = a * b;
    return c;
}

int main() {
    cout << multiply(2, 3) << endl;
    return 0;
}

```

Handwritten notes:
 - A box with value 6 is labeled `c`.
 - Variables `a` and `b` are boxed with values 2 and 3 respectively.
 - An arrow labeled "copy" points from the boxed `a` and `b` to the boxed `c`.
 - A bracket under `multiply(2, 3)` in `main` is labeled "copy".
 - The output `// 6` is shown.

```

int multiply(int a, int b) {
    static int c = a * b; // static variable
    return c;
}

int main() {
    cout << multiply(2, 3) << endl;
    return 0;
}

```

Handwritten notes:
 - A box with value 6 is labeled `c`.
 - Variables `a` and `b` are boxed with values 2 and 3 respectively.
 - An arrow labeled "copy" points from the boxed `a` and `b` to the boxed `c`.
 - A bracket under `multiply(2, 3)` in `main` is labeled "copy".
 - The output `// 6` is shown.

- We say an expression is a **Lvalue expression** if it evaluates to an **entity that has an identity** and persists beyond the end of the expression — for e.g. **a variable**
- We say an expression is a **Rvalue expression** if it evaluates to **literals** or **values** returned by **functions/operators** that are discarded at the end of the expression.

```
int f(int a, int b) {
    return a + b;
}

int main() {
    int x = 2; // 2 is an rvalue expr
    const double d = 3.14; // 3.14 is an rvalue expr

    int y = x; // x is a modifiable (non-const) lvalue expr
    const double e = d; // d is a non-modifiable (const) lvalue expr

    int z = f(x, y); // f(x, y) is an rvalue expression

    int w = x + 1; // x + 1 is an rvalue expression
    int q = static_cast<int>(d); // the o/p of static casting is an rvalue expr

    return 0;
}
```

- **Lvalue expressions** implicitly convert to **rvalue expressions** in contexts where an **rvalue** is expected but an **lvalue** is provided.

```
int c = 1; // global variable

int f() {
    return c;
}

int main() {
    f() = 0;
    cout << c << endl;
    return 0;
}
```

```
int c = 1; // global variable

int& f() {
    return c;
}

int main() {
    f() = 0;
    cout << c << endl;
    return 0;
}
```

Function Declaration vs Function Definition

In C++, you cannot call a function before you either **declare** it or **define** it.

fn
defn

ret-type fn-name (.....) {

 }

ret-type fn-name(.....); // fn decl^r // fn prototype

Function Overloading

In C++, we cannot create multiple functions with the **same signature** (function name + parameters list) however, we can **overload** functions.

```

int f(int a, int b) {
    return a+b;
}

int f(int a, int b) {
    return a+b;
}

int main() {
    cout << f(2, 3) << endl; // ambiguous fn call
    return 0;
}

```

Function overloading means being able to create multiple functions with the **same name** as long as the **parameter list** differ either in terms of parameter types or the no. of parameters.

```

double areaOfTriangle(int b, int h) {
    return 0.5 * b * h;
}

double areaOfTriangle(int a, int b, int c) {
    double s = (a+b+c)/2.0;
    return sqrt(s*(s-a)*(s-b)*(s-c));
}

int main() {
    cout << areaOfTriangle(4, 5) << endl;
    cout << areaOfTriangle(3, 4, 5) << endl;
    return 0;
}

```

In C++, functions cannot be overloaded based on the **return type**.

```

int f(int a, int b) {
    return a+b;
}

double f(int a, int b) {
    return a+b / 2.0;
}

int main() {
    cout << f(2, 3) << endl; // ambiguous fn call
    return 0;
}

```

Namespaces

A **namespace** is a mechanism used by C++ to disambiguate conflicting identifiers. Formally, it is a region that allows you to declare names inside of it for the purpose of **disambiguation**.

```

namespace add {
    int f(int a, int b) {
        return a+b;
    }
}

namespace mul {
    int f(int a, int b) {
        return a*b;
    }
}

int main() {
    cout << add::f(2, 3) << endl; // 5
    cout << mul::f(2, 3) << endl; // 6
    return 0;
}

```

*f(int a, int b) {
return a/b;
}*

*s+d:: cout
s*d:: cin
.*

f(2,3);

All the names within a namespace must be **unique** to avoid **naming collisions**.

[R][HW] Function Templates

```
int max(int a, int b) {
    return (a < b) ? b : a;
}

int main() {
    cout << max(2, 3) << endl;
    return 0;
}
```

```
double max(double a, double b) {
    return (a < b) ? b : a;
}

int main() {
    cout << max(1.732, 1.414) << endl;
    return 0;
}
```

```
template <typename T>
T max(T a, T b) {
    return (a < b) ? b : a;
}

int main() {
    cout << max<int>(2, 3) << endl;
    cout << max<double>(1.732, 1.414) << endl;
    return 0;
}
```

Default Arguments in Functions

In C++, we can provide **default arguments** i.e. **values** to **function parameters**, such parameters are known as **default parameters**. During a call to a function if we **don't pass** the **value** for a default parameter then its default value is used.

```
int add(int w, int x, int y=0, int z=0) {
    return w+x+y+z;
}

int main() {
    cout << add(1, 2) << endl;
    cout << add(1, 2, 3) << endl;
    cout << add(1, 2, 3, 4) << endl;
    return 0;
}
```

- In C++, default parameters must provided from right to left.

```
int add(int w=0, int x=0, int y, int z) {
    return w+x+y+z;
}

int main() {
    cout << add(2, 3) << endl;
    return 0;
}
```

- In C++, we cannot overload functions on default parameters.

```
int f(int x, int y=0,) {
    return x + y;
}

int f(int x, double y=0) {
    return x * y;
}

int main() {
    cout << f(2) << endl; // ambiguous fn call
    return 0;
}
```

$f(int\ a, int\ b, int\ c=0, int\ d=0) \{$
 $\quad ret\ a+b+c+d;$
 $\}$
 $f(2, 3, 4, 5); // 14$
 $f(2, 3, 4); // 9$
 $f(2, 3); // 5$